

---

# Decision Support Databases 25-26

## Notes

---

# Contents

|          |                                               |           |
|----------|-----------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                           | <b>2</b>  |
| 1.1      | Information Systems . . . . .                 | 3         |
| 1.2      | Data Warehouses . . . . .                     | 3         |
| 1.2.1    | Data Warehousing Architectures . . . . .      | 4         |
| 1.2.2    | What to Model . . . . .                       | 5         |
| <b>2</b> | <b>Modeling for Operational Databases</b>     | <b>8</b>  |
| 2.1      | Information Modeling . . . . .                | 8         |
| 2.1.1    | What to Model . . . . .                       | 9         |
| 2.2      | Data Models . . . . .                         | 10        |
| 2.2.1    | Object Data Model . . . . .                   | 10        |
| 2.2.2    | Relational Data Model . . . . .               | 13        |
| <b>3</b> | <b>Modeling for Data Warehouses</b>           | <b>17</b> |
| 3.1      | Data Models . . . . .                         | 17        |
| 3.1.1    | Dimensional Fact Model . . . . .              | 17        |
| 3.1.2    | Multidimensional Relational Model . . . . .   | 21        |
| 3.2      | Design Approaches . . . . .                   | 22        |
| <b>4</b> | <b>Customer Relationship Management</b>       | <b>29</b> |
| 4.1      | Types of Analyses . . . . .                   | 29        |
| <b>5</b> | <b>Multidimensional Data Cube Model</b>       | <b>32</b> |
| 5.1      | OLAP Operations in the Data Cube . . . . .    | 32        |
| 5.2      | The Extended Cube . . . . .                   | 33        |
| <b>6</b> | <b>On-line Analytical Processing Analysis</b> | <b>35</b> |
| 6.1      | OLAP Systems Solutions . . . . .              | 35        |
| 6.2      | Reports . . . . .                             | 36        |
| 6.2.1    | Simple Reports . . . . .                      | 37        |
| 6.2.2    | Moderately Difficult Reports . . . . .        | 40        |
| 6.2.3    | Very Difficult Reports . . . . .              | 41        |

# Chapter 1

## Introduction

In modern society, organizations accumulate large amounts of data, stored and managed by computers via a DBMS (Database Management System). However, this data is often underutilized, and important decisions are still based on intuition rather than informed decisions derived from data analysis. Decision support information systems are specific types of information systems designed for summarizing large amounts of data in a form that is easy to interpret. Analysis performed by such systems goes beyond simple queries executed by traditional DBMSs on operational databases, which are optimized for on-line transaction processing. Organizations maintain a separate database, called data warehouse, specifically organized for decision support applications.

### Data, Information, Knowledge

**Data** is a representation of facts, concepts, or instructions without context, which can be processed by computers.

**Information** is data (raw or in a condensed form) interpreted within a certain context.

**Knowledge** is information that expands the recipient's capability of understanding reality, allowing them to make predictions, decisions, and take actions.

Each organization has a set of specific goals to pursue, and to achieve these goals, it uses a **structure**, a set of **resources**, and a set of **processes** that transform resources into goods and services. Processes can be classified in three categories according to the **Anthony tri-angle**: **strategic activities** (for long-range planning), **tactical activities** (for short term planning and control), and **operational activities** (for day to day operations). For any process, information is a key resource; different organizations will have different information needs, depending on their structure, resources, and processes. Information systems are designed to satisfy these needs.

## 1.1 Information Systems

### Information System

An **information system** is an organized collection of resources, people, and procedures finalized to collect, store, process, and communicate the information needed to support the on-going activities of an organization.

A computerized information system is a type of information system where information is stored and processed by a computer. From now on, we will simply refer to computerized information systems as information systems.

Information systems can be classified into three categories:

- **Operational:** data is organized in a database, which is managed by a traditional DBMS and interacted with using transactions. This type of system is used for day-to-day operations;
- **Decision Support:** data is organized in a dedicated data warehouse, managed by a specialized DBMS. This type of system is used for Business Intelligence applications, i.e., to search for useful insight to support decision making. DSS have been developed in the late 70's to help managers in making decisions of three types:
  - **Structured:** for which a well-defined decision-making procedure exists;
  - **Unstructured:** for which a well-defined decision-making procedure does not exist, so that the decision will only depend on manager experience;
  - **Semi-structured:** when the decision-making procedure is partly defined, so it also requires manager input.
- **Web-based:** for E-commerce, to bring the business to the Web.

## 1.2 Data Warehouses

### Data Warehouse

A **data warehouse** is a subject-oriented, integrated, nonvolatile, and time-varying collection of data in support of management's decisions.

A data warehouse is a specialized type of database that stores data by subject, not by applications (*subject-oriented*), includes data from multiple heterogeneous sources and merged in a coherent form (*integrated*), is not changed interactively via transactions, but is updated periodically to add or delete data (*nonvolatile*), and it contains historical data that describes the state of the enterprise at different points in time, unlike operational databases where data is kept up-to-date (*time-varying*). A special type of data warehouse is the **data mart**, which is focused on data regarding only one subject and is usually smaller in size.

Three types of decision support can be provided. **Reports**, which are simple presentations of data in a structured format, **multidimensional analysis**, which also allows users to interactively explore data from different perspectives, and **data mining/exploratory data analysis**, where algorithms are used to automatically discover interesting patterns or relationships in data.

The reasons for separating operational databases from data warehouses are mainly two: first, complex data analysis queries would degrade the performance of operational DBMSs, which are optimized for on-line transaction processing; second, the two systems have different structures, contents, and uses, since operational databases do not store any historical data and do not integrate different sources. Traditional applications that use databases are called **OLTP (On-Line Transaction Processing)**, while decision support applications that use data warehouses are called **OLAP (On-Line Analytical Processing)**.

### 1.2.1 Data Warehousing Architectures

The process of bringing data from operational database sources into a single data warehouse is called **data warehousing**. In practice, three types of solutions are adopted.

**Single-Layer Architecture** There is only one layer of data managed by the operational system. The data warehouse is defined as a view on the operational database, but is not a physically separate component. This solution is very low-cost and easy to implement, and as such is often used by small organizations. Of course, it does not actually separate operational and analytical operations, so it is not suitable for large organizations.

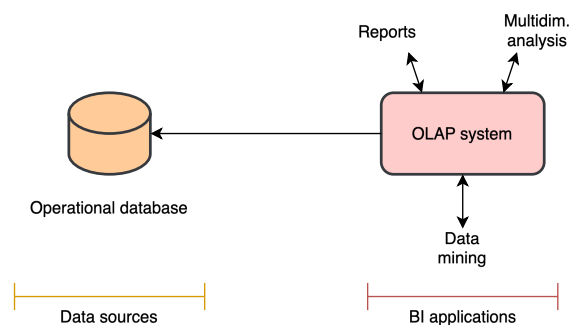


Figure 1.1: Single-Layer Architecture.

**Two-Layer Architecture** The data warehouse is a separate component from the operational database and is managed by its dedicated DBMS. The warehouse is loaded with data via **ETL (Extract, Transform, Load)** applications from the operational database and any other external source, which also brings this data in a consistent format and updates it periodically.

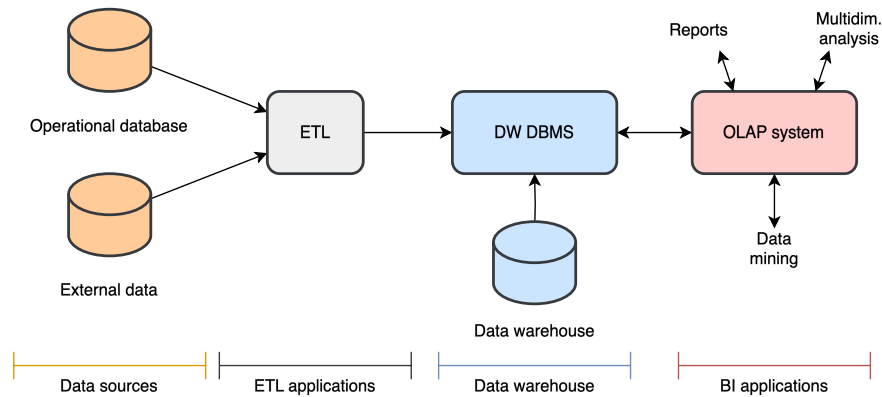


Figure 1.2: Two-Layer Architecture.

**Three-Layer Architecture** The third layer is called **data staging**. It contains data obtained via integration in the ETL process and prepares it to be loaded into the data warehouse, and may be implemented at different levels of complexity (it can be as simple as a set of files and as complex as a fully developed relational database). In this solution, extraction and integration is separated from reorganization and loading.

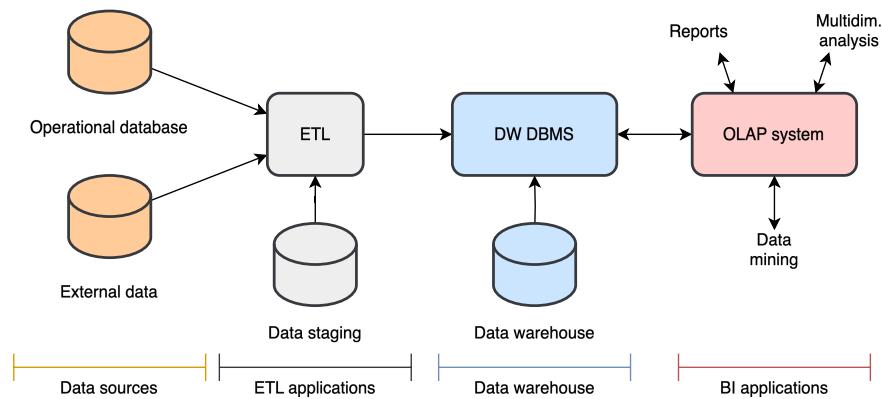


Figure 1.3: Three-Layer Architecture.

## 1.2.2 What to Model

Data must be organized taking into account how managers will use it to support their decisions about the performance of key business processes. More precisely, the relevant informa-

tion to model are:

- **Facts:** managers are interested in analyzing facts (i.e., observations) about the performance of a process;
- **Measures:** facts are accompanied by numerical attributes called measures (such as quantities, sizes, costs, revenues, etc.);
- **Dimensions:** managers think in terms of business dimensions, which give facts a context to make them meaningful. A dimension is often described by a set of attributes. If this is not true, it is called a *degenerate dimension*;
- **Metrics:** measures are analyzed via summaries called metrics, obtained by applying aggregation functions over facts by certain dimensions or combinations of dimensions. Common metrics are about economic performance, but when they relate to efficiency and quality they are called **Key Performance Indicators (KPI for short)**.

Aggregation functions are classified in three categories:

- **Distributive:**  $sum()$ ,  $count()$ ,  $max()$ ,  $min()$ . An aggregate function  $f$  on a multiset  $V$  is distributive if there is a local aggregate function  $f_l$  and a global aggregate function  $f_g$ , such that for any  $k$ -partition  $\{V_1, \dots, V_k\}$  of  $V$  we have  $f(V) = f_g(\{f_l(V_1), \dots, f_l(V_k)\})$ .
  - **Algebraic:**  $avg()$ ,  $stddev()$ ,  $var()$ . An aggregate function is algebraic if it can be computed using a finite algebraic expression defined over distributive functions.
  - **Holistic:**  $median()$ ,  $mode()$ ,  $rank()$ . An aggregate function is holistic if it cannot be computed from other aggregate functions.
- **Hierarchies:** managers are also interested in analyzing metrics at different levels of detail, by exploiting the fact that certain dimensions have a set of attributes that form a hierarchy (e.g., date can be broken down into year, semester, quarter, month, day).

A hierarchy always implies a functional dependency between the attributes involved.

### Functional Dependency

Given a relation schema  $R$  and  $X, Y$  subsets of attributes of  $R$ , a functional dependency  $X \rightarrow Y$  is a constraint that specifies that for every possible instance  $r$  of  $R$  and for any two tuples  $t_1, t_2 \in r$ ,  $t_1[X] = t_2[X]$  implies  $t_1[Y] = t_2[Y]$ .

In simpler terms, a functional dependency  $X \rightarrow Y$  means that any time we know the value that a tuple takes in  $X$  we can uniquely determine the value it takes in  $Y$ . In the date

example, there is the following functional dependency:  $\text{day} \rightarrow \text{month} \rightarrow \text{quarter} \rightarrow \text{semester} \rightarrow \text{year}$ . Note that in order for the hierarchy to hold, the day attribute must also include the specific month and year, the month attribute must include the year, and so on.



# Chapter 2

## Modeling for Operational Databases

This chapter discusses modeling methods and techniques for operational databases, introducing the main notation elements of the most common models.

### 2.1 Information Modeling

When a database system is created, it is based on some model that reproduces the essential characteristics of the real world information it is intended to represent. In general, models can be either *iconic*, if they retain physical characteristics of the entities they represent, or *symbolic*, if they use symbols to represent entities and their relationships. In the context of information systems, symbolic models are used.

#### Symbolic Model

A **symbolic model** is a subjective, formal representation of ideas and knowledge about some aspects of the real world, called domain of discourse, designed to serve a specific purpose.

A model simplifies reality by focusing on what is important for the task at hand, and uses a formal language to express this simplified representation.

Modeling in databases is broken down into three levels of abstraction:

- **Conceptual level**, in which the basic structure of the database is defined at an high-level, after analyzing the problem and considering any user requirement. Some examples of conceptual models are the **Entity-Relationship (ER)** model and the **Object Data Model (ODM)**;
- **Logical level**, where the conceptual model is mapped to a more detailed description of the data structures, including all attributes and constraints. The model is still

independent from any specific DBMS. An example of logical model is the **Relational Data Model**;

- **Physical level**, where the physical data structures are defined for the elements in the logical model. The choices done at this level must take in consideration the features of the DBMS used to implement the database.

### 2.1.1 What to Model

The reality to be modeled is considered in terms of concrete knowledge, abstract knowledge, procedural knowledge, dynamics, and communications.

**Concrete Knowledge** Concrete knowledge refers to specific facts known of the reality to be represented. We can assume reality consists of *entities*, *properties*, and *relationships*.

#### Entity, Property, Relationship

An **entity** is anything for which certain facts should be recorded, independently of the existence of other entities.

A **property** is a fact about an entity which is not meaningful in itself, but only because it describes an entity of interest.

A **relationship** is a fact which correlates independent entities.

For example, we want to model the administration system of a library. Examples of entities can be a book, a bibliographic description, a registered user, or a loan record. Properties may be a user's name and address, or a book's title and author. Entities with the same properties are said to have the same **type**, and are classified in the same **collection**, or **entity set**. The books titled "1984", "Moby Dick", and "The Great Gatsby" are all part of the same entity set, and are all of type *book*. As for relationships, examples could be the relationship between a bibliographic description and one or more books (the same book may be owned multiple times by the library), or between a user and their loan records.

**Abstract Knowledge** Abstract knowledge concerns facts which impose certain restrictions on admissible values of concrete knowledge and on the way these values can evolve over time, or expresses rules on how to derive new information. These rules are called **integrity constraints**. In the library example, abstract knowledge could include rules about the data type of attributes, such that book properties *title* and *author* must be strings, while *length* must be a non-negative integer; another example may be the rule that a loan can only be borrowed for two weeks at a time, and must be associated with the *id* of a registered user; the age of a user can be automatically calculated by subtracting the *birthdate* from the current

date. Relationships can also have constraints that limit the possible correlated entities. The most important ones are **structural constraints**, which are:

- **Cardinality**, *one* or *many*, to specify how many entities of one collection can be associated with entities of another collection, e.g., a bibliographic description can be associated with *many* book copies, while a book copy is associated with only *one* bibliographic description, so the relationship is called *one-to-many*;
- **Participation**, *total* or *partial*, to specify whether an entity of one collection can have entities of another collection associated with it, e.g., a loan record must be associated with one registered user, so the participation from the loan record side is *total*; on the other hand, a registered user may have never loaned a book, so the participation from the registered user side is *partial*.

## 2.2 Data Models

To construct the conceptual model for an operational database, we define a **schema**, a collection of time-invariant definitions which model the structure of admissible data (including integrity constraints), and procedural knowledge (i.e., the elementary operations applied to concrete knowledge to cause changes). Different formalisms, each with a specific data model, can be used to define the schema.

### Data Model

A **Data Model** is a set of abstraction mechanisms, with associated operators and implicit integrity constraints, used to define a database schema.

For operational databases, we will see the Object Data Model and the Relational Data Model, for the conceptual and logical levels respectively.

### 2.2.1 Object Data Model

The basic mechanisms of the ODM are:

- **Objects**: they are the equivalent of entities. An object is a software entity with an internal state represented by instance variables, and a behaviour described by a set of methods. Each object has its own identity that persists over time, even if its internal state change;
- **Object types**: just like an entity belongs to a type, an object belongs to an object type. An object type describes the state fields and the implementation of methods common to all objects of that type;

- **Classes:** a modifiable sequence of objects with the same type. They are the equivalent of entity sets. Each class introduces the definition of a type and a constructor for values of this type, as well as a name to denote the class itself;
- **Relationships:** classes have relationships with other classes, which are characterized by a cardinality and partiality. A relationship may also have attributes, or be recursive (i.e., a class has a relationship with itself);
- **Inheritance:** a mechanism that allows something to be defined by only describing how it differs from a previously defined one. It is distinct from subtyping, which is a relation between types such that when one type  $A$  is a subtype of another type  $B$ , then any operation applicable to type  $B$  can also be applied to type  $A$  (although inheritance is one way to define subtypes). A type can be defined from a single supertype (*single inheritance*) or from several supertypes (*multiple inheritance*).
- **Class hierarchies:** similarly to types, classes can also be organized in hierarchies. If  $C_1$  is a subset of  $C_2$ , then  $C_1$  is a *subclass* of  $C_2$ , and this implies that:
  - The type of elements in  $C_1$  is a subtype of the type of the elements in  $C_2$ ;
  - The elements in  $C_1$  are a subset of the elements in  $C_2$ , and  $C_1$  inherits any relationship defined on  $C_2$ .

When the same superclass is shared by two or more subclasses, two constraint can be defined: **overlap** and **covering**. A no overlap constraint specifies that two subclasses cannot share any element. In the absence of this constraint, the same object may belong to more than one subclass. A covering constraint specifies that the objects in the subclasses must collectively include all elements in the superclass. In the absence of this constraint, the superclass may have elements that do not belong to any subclass. When both constraints are present, we call the hierarchy a **generalization**.

The following figures illustrate a possible graphical notation used for ODM. There is no standard notation; the one used here is based on the notation used in UML.

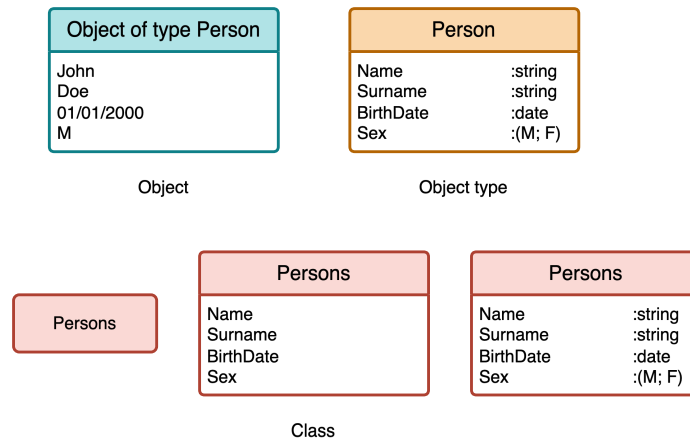


Figure 2.1: Graphical notation for objects, object types, and classes. Classes can be shown with different levels of detail.

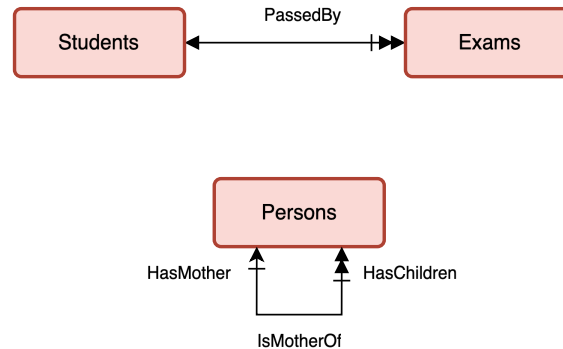


Figure 2.2: Simple relationships between classes. Each relationship is represented by an arc with a label. Multivalued relationships are represented with a double arrow. Optional relationships have a crossed line. Recursive relationships require labels on the ends of the arcs to clarify the meaning.

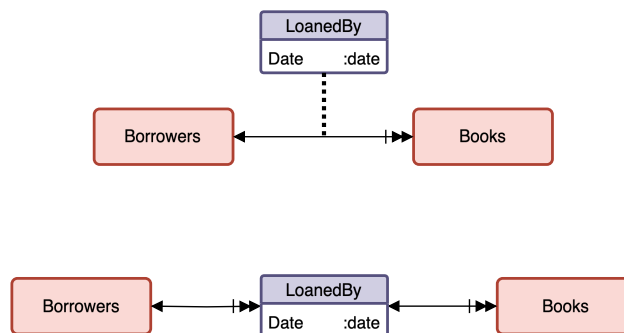


Figure 2.3: Relationships with attributes.

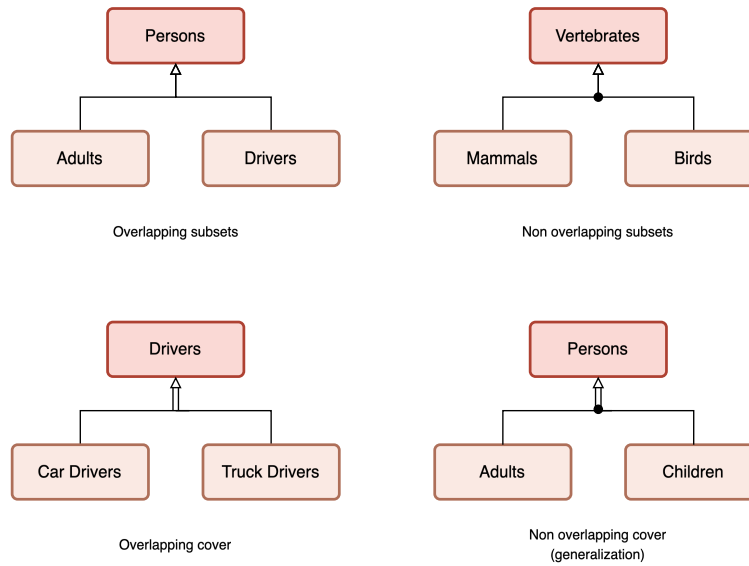


Figure 2.4: Class hierarchies.

## 2.2.2 Relational Data Model

The Relational Data Model describes databases in terms of sets of **tuples** (also called **records**), and associations among elements are expressed in terms of values of their attributes.

### Relational Database

A **relational database** is described by a set of relation schemas  $R : \{T\}$  defined as follows:

- **integers**, **floats**, **booleans**, and **strings** are primitive types.
- If  $T_1, \dots, T_n$  are primitive types, and  $A_1, \dots, A_n$  are distinct attribute names, then  $T = (A_1 : T_1, \dots, A_n : T_n)$  is a **tuple type** of degree  $n$ . The attribute order is unimportant.
- If  $T$  is a tuple type, then  $\{T\}$  is a **relation type**.
- A **relation schema**  $R : \{T\}$  is a variable  $R$  with a relation type.

For brevity, instead of writing  $R : \{T\}$ , we will use  $R(T)$ . Also, when the type of the attributes in a schema is unimportant, we will simply write it as  $R(A_1, \dots, A_n)$ .

### Tuple

A **tuple**  $(A_1 := V_1, \dots, A_n := V_n)$  of type  $T = (A_1 : T_1, \dots, A_n : T_n)$  is a set of pairs  $(A_i, V_i)$  with  $V_i$  of primitive type  $T_i$ .

## Relation

A **relation** (or schema instance) is a finite set of tuples with the same type  $T$ .

A relation does not contain any duplicate tuples, since it is a set and not a multi-set. The order of relation elements and attributes is unimportant.

The following statements hold true:

- Two tuple types are equal if and only if they have the same degree, same attributes, and same type of the attributes with the same name.
- Two relation types are equal if and only if they have the same tuple types.
- Two tuples of the same type are equal if and only if they have the same set of attribute-value pairs.
- Two relations of the same type are equal if and only if they have equal tuples.

## Key

A **key** for a relation is the minimal subset of attributes whose values identify a tuple. Out of all the possible keys, the database designer chooses a **primary key** (usually the shortest).

Relationships between relations are expressed using **foreign keys**, which take as values those of the primary key of another relation. For example, consider the two relations:

***Students***(*StudentCode*, *Name*, *City*, *BirthYear*)  
***Exams***(*Subject*, *Candidate*, *Date*, *Grade*)

where attribute *StudentCode* is the primary key of *Students*, and the pair *Subject*, *Candidate* is the primary key of *Exams*. Since the attribute *Candidate* takes values that match those of *StudentCode*, it is a foreign key.

**Relational Algebra** Relational algebra is a set of operators on relations whose results are also relations. Expressions in relational algebra are called **queries**, and the standard language used by DBMSs to write queries is SQL (Structured Query Language): it is a declarative language, meaning that the user specifies what data to retrieve, and not how to retrieve it. There are six fundamental operations, plus additional operations that can be derived from them. These operations are:

- **Rename**  $\rho_{A_1 \leftarrow B_1, \dots, A_m \leftarrow B_m}(R)$ : assigns a new name to attributes.  $A_1, \dots, A_m$  are attributes of relation  $R$ , and  $B_1, \dots, B_m$  are not. The result is a relation with type  $\{(B_1 : T_1, \dots, B_m : T_m)\}$ , whose tuples are those of  $R$  with renamed attributes.
- **Project**  $\pi_{A_1, \dots, A_m}(R)$ : selects a subset of attributes from the relation  $R$ , deleting any duplicate. The result is a relation with type  $\{(A_1 : T_1, \dots, A_m : T_m)\}$ , whose tuples are those of  $R$  restricted to attributes  $A_1, \dots, A_m$ .
- **Generalized project**  $\pi_{exp_1 \text{ AS } B_1, \dots, exp_n \text{ AS } B_n}(R)$ : a combination of projection and renaming that also allows the output to contain the result of expressions.  $exp_1, \dots, exp_n$  are arithmetic expressions that involve constants and attributes of  $R$ , and  $B_1, \dots, B_n$  are attribute names. The result is a relation with type  $\{(B_1 : T_{exp_1}, \dots, B_n : T_{exp_n})\}$ .
- **Select**  $\sigma_{cond}(R)$ : selects the tuples in  $R$  that satisfy the condition  $cond$ . This condition is a propositional logic expression ( $\wedge, \vee, \neg$ ) over comparison predicates ( $\leq, <, \geq, >, =, \neq$ ) and arithmetic expressions ( $+, -, *, \div$ ).
- **Grouping**  $_{A_1, \dots, A_n} \gamma_{f_1, \dots, f_k}(R)$ : returns a relation whose tuples are grouped by the values of the specified attributes, and applies the specified aggregate functions to the remaining attributes.  $A_1, \dots, A_n$  must be attributes of  $R$ , and  $f_1, \dots, f_k$  are aggregate functions (min, max, count, sum, average, etc.) calculated over one or more of the grouped attributes. The relation type of the result is  $\{(A_1 : T_1, \dots, A_n : T_n, f_1 : T_{f_1}, \dots, f_k : T_{f_k})\}$ .
- **Generalized grouping**  $_{A_1, \dots, A_n} \gamma_{f_1 \text{ AS } F_1, \dots, f_k \text{ AS } F_k}(R)$ : extends the grouping operation by allowing the renaming of attributes obtained via aggregation functions. The relation type of the result is  $\{(A_1 : T_1, \dots, A_n : T_n, F_1 : T_{f_1}, \dots, F_k : T_{f_k})\}$ .
- **Union**  $R \cup S$ : returns the union of the two sets of tuples in  $R$  and  $S$ , given that they have the same relation type.
- **Set difference**  $R - S$ : returns the set of tuples in  $R$  that are not also in  $S$ , given that they have the same relation type.
- **Product**  $R \times S$ : given  $T$  of type  $\{(A_1 : T_1, \dots, A_n : T_n)\}$  and  $S$  of type  $\{(A_{n+1} : T_{n+1}, \dots, A_{n+m} : T_{n+m})\}$  with disjoint set of attributes, the product operator returns a relation of type  $\{A_1 : T_1, \dots, A_n : T_n, A_{n+1} : T_{n+1}, \dots, A_{n+m} : T_{n+m}\}$ , whose tuples are all possible combinations of tuples in  $R$  and  $S$ .
- **Join**  $R \bowtie_{A_i=A_j} S$ : combines tuples from  $R$  and  $S$  that have the same values for the attributes  $A_i$  of  $R$  and  $A_j$  of  $S$ . The two relations have type  $\{(A_1 : T_1, \dots, A_n : T_n)\}$  and  $\{(A_{n+1} : T_{n+1}, \dots, A_{n+m} : T_{n+m})\}$ , and have a disjoint set of attributes. This operation is equivalent to calculating  $\sigma_{A_i=A_j}(R \times S)$ .



- **Intersection**  $R \cap S$ : returns the set of tuples that are present in both  $R$  and  $S$ , given that they have the same relation type.
- **Natural join**  $R \bowtie S$ : a special case of join that is only applicable when the two relations have attributes with the same name. It combines tuples in  $R$  and  $S$  that have the same values for all attributes with the same name in both.
- **Division**  $R \div S$ : let  $XY$  be the attributes of  $R$ , and  $Y$  the attributes of  $S$ . Then, the result of the division between  $R$  and  $S$  is a relation with attributes  $X$  such that

$$W = \{w | \forall t \in S. (w \circ t \in R)\}$$

where  $\circ$  is the concatenation operator. In other words, it returns all the values of attributes  $X$  in  $R$  which appear in a tuple with all values of attributes  $Y$  in  $S$ .

If  $S \times W = R$ , then  $R \div S = W$ .

There are also a series of **equivalence rules** to simplify expressions:

$$\begin{aligned}
\pi_A(\pi_{A,B}(R)) &\equiv \pi_A(R) && \text{(Cascading of projections)} \\
\sigma_{cond_1}(\sigma_{cond_2}(R)) &\equiv \sigma_{cond_1 \wedge cond_2}(R) && \text{(Cascading of selections)} \\
\sigma_{cond_R \wedge cond_S}(R \bowtie S) &\equiv \sigma_{cond_R}(R) \bowtie \sigma_{cond_S}(S) && \text{(Commutativity of selection and join)} \\
R \bowtie (S \bowtie T) &\equiv (R \bowtie S) \bowtie T && \text{(Associativity of join)} \\
R \bowtie S &\equiv S \bowtie R && \text{(Commutativity of join)} \\
\sigma_{cond_X}(X \gamma_F(R)) &\equiv_X \gamma_F(\sigma_{cond_X}(R)) && \text{(Selection distributes over grouping)}
\end{aligned}$$

# Chapter 3

## Modeling for Data Warehouses

This chapter will illustrate the main design approaches and methodologies to create a data warehouse, and introduce the data models used to represent its structure.

### 3.1 Data Models

This section will introduce three models to define the structure of a data warehouse: for the conceptual level, the **Dimensional Fact Model (DFM)**, while for the logical level, the **Multidimensional Relational Model** and the **Multidimensional Cube Model** (which will be discussed in a later chapter).

#### 3.1.1 Dimensional Fact Model

A data warehouse is based on a multidimensional model, i.e., a model that takes in consideration multiple business dimensions to allow the user to explore aggregated information across many possible dimensional combinations to get a full perspective on the data. The Dimensional Fact Model is a graphical conceptual model for data warehouses. The following are its basic elements.

**Facts** The most important abstraction mechanism of the conceptual model is the collection of facts, which are observations of the performance of a business model. Facts are modeled as rectangles divided in two parts: the top contains the fact name, while the bottom contains a set of **measures**. A measure is a numerical property of a fact that describes one of its quantitative aspects of interest for analysis. Facts that are not associated with any measure are called *factless fact*, although to keep terminology consistent, we will call them **measureless facts**. This type of fact is defined when there is only interest in counting its number of instances.

Facts can be of different nature:

- **Transaction facts** represent information on a specific event that occurred at a specific point in time during the execution of a business process, e.g., a withdrawal on a bank account.
- **Periodic snapshot facts** represent the information on a series of events that have occurred over a period of time, e.g., the monthly summary of all transactions on a bank account.
- **Accumulating snapshot facts** represent the information on the lifetime of an evolving event that has a duration and a change over time, e.g., a mortgage application which is done in several steps (presenting the documents, undergoing bank review, approval/refusal, completion).

Also measures can be classified in different types:

- **Additive/flow/rate measures** can be meaningfully aggregated with the function *sum* by any dimension. This is the most common type of measure. An example may be the number of products sold in a day.
- **Semi-additive/stock/level measures** can be meaningfully aggregated with the function *sum* by certain dimensions, but not all. These measures refer to a particular point in time and are used to record the state of the event: for example, the monthly inventory quantity of a certain product. We say that a measure is semi-additive with respect to a dimension  $D_1$  if it cannot be aggregated with the function *sum* for groups of data with different values of  $D_1$ .
- **Non-additive/value-per-unit measures** cannot be aggregated with the function *sum* by any dimension. These measures are usually the result of ratios, so the only meaningful operation is counting the number of facts with a specific value. Other common non-additive measures are per-unit prices, measures of intensity (e.g., a temperature), or averages.
- **Calculated measures** are derived from other measures. These are used to avoid having users perform these calculations, such as calculating revenue from product price minus discount.



Figure 3.1: Graphical representation of facts.

**Dimensions** Dimensions give facts a context. Graphically, they are represented by lines starting from the corresponding fact and ending with a circle. A dimension may also be described by a set of attributes, which are drawn as lines ending with a circle connected to the “main” circle of the dimension. Each attribute is also labeled with its unique name, and the same name should not be used for attributes of different dimensions. A particular type of dimensional attribute are **descriptive attributes**: they are not directly used in aggregation, but are still included in the dimension table to keep track of useful information.

In some cases, an interesting aspect to model is a hierarchical relationship between attributes. Using relational data model terminology, a **hierarchy** can be defined when there is a functional dependency between attributes: for example, if we define the dimension *Date*, with attributes *Day*, *Month*, *Year*, then we can define the hierarchy  $Day \rightarrow Month \rightarrow Year$ . The same attribute may even appear in multiple hierarchical structures for the same dimension: if we also add the attribute *Week*, then we can define  $Week \rightarrow Year$  (this is because a week can overlap multiple months, so it does not belong to the previous dependency chain). Graphically, the attributes in the hierarchy are connected via directed edges that follow the direction of the functional dependency.

Hierarchies may be **recursive**, when it involves the same dimension. For example, an *employee* may have an assigned role that puts them in charge of other employees, as well as being managed by someone else; this relationship can be expressed via the *supervisor* attribute, which allows the creation of a hierarchy of employees in a manager-subordinate relationship. Recursivity is represented by a crossed arrow that loops back in the same dimension. Hierarchies can also be **ragged**, if one or more elements in the chain are optional. For example, an hierarchy can be built with the attributes *City*, *State*, and *Country*. Since some cities may belong to countries without states, *State* can be set as optional. Graphically, the optional attributes in a ragged hierarchy are crossed by a line. Attributes and hierarchies can be **shared** among multiple dimensions. These attributes are represented by a double circle, and their incoming arc are always directed (regardless of whether they belong to a hierarchy or not).

A dimension may also be **multivalued**, if the same fact involves multiple instances of the same dimension. For example, a product order is managed by two agents, or a medical treatment involves multiple doctors. Multivalued dimensions are represented by a double arrow head exiting from the fact table. A dimension without attributes is called **degenerate**. They are simply represented by a line exiting the fact table and a singular circle at the end without any additional attributes. If a dimension is **optional**, it is represented by a dashed line.

Although not explicitly modeled at this level, the concept of changing over time is typically reported in the documentation, to be considered in the next phases. **Changing dimensions** are those dimensions whose attribute values may change over time. For example, a customer’s address, phone number, or marital status; a student’s enrollment status or major; a product’s

price or discount. There are four types of changing dimensions, the first three of which are **slowly changing dimensions**, and the last one being the **fast changing dimension**.

- **Type 1: overwriting the history.** When a dimensional attribute is updated, only the latest value is held in the data warehouse. This solution does not preserve the sequence of changes each attribute undergoes over time, potentially leading to a loss of context. For example, imagine a *Customer* dimension that contains various attributes, including the address. One customer changes residence from Rome to Milan, all sales that involved this customer will now consider the new Milan address, even if they occurred before the address change.
- **Type 2: preserving all history.** Both old and new values of the attribute are held, without changing the structure of dimensions. In practice, this means that each value change produces a new record in the data warehouse.
- **Type 3: preserving one or more versions of history.** If a dimensional attribute changes value, the structure of the dimension is extended with one or more additional attributes to keep track of different versions, as well as a timestamp that stores the date of the change. In the address change example, a new attribute *OldAddress* is created to keep the previous value with the Rome address, and the *address* attribute is updated with the Milan one. This solution is seldom used.
- **Type 4: fast changing.** The dimensional attribute changes very frequently (e.g., a status that is updated each few hours or minutes).

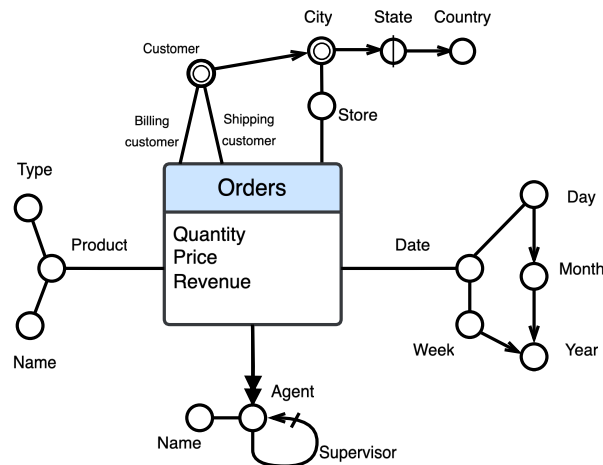


Figure 3.2: Graphical representation of dimensions.

The key steps to design a DFM schema are the following:

1. Gather requirements, identifying the key elements to insert in the model.

2. Identify the granularity of the fact, i.e., the level of detail at which the fact will be represented. Granularity determines which measures and dimensions will appear in the model. As a general rule, it is best to choose a fine grain, even if it increases the number of facts to be treated, since later on, aggregation functions can always reduce the level of detail.
3. Identify the fact measures. Not everything that is numeric is necessarily a measure.
4. Identify the dimensions and dimensional attributes. To discern dimensions in the original requirements analysis, it is useful to consider the classic 5W-1H questions.
5. If possible, identify any dimensional attribute hierarchy.

### 3.1.2 Multidimensional Relational Model

A conceptual model is transformed into a relational logical schema by applying a set of mapping rules. The main relational DBMS vendors provide OLAP servers that map operations on multidimensional data to standard relational operations on specialized relational DBMS to store and manage data warehouses. Such servers are called **ROLAP** (relational OLAP).

The basic elements of logical schemas are the **fact table** and the **dimension table**: a fact table stores all the measures of the corresponding fact and any necessary foreign keys to the dimension tables, while a dimension table stores all the attributes of the corresponding dimension, and eventually foreign keys to other dimension tables.

Schemas can follow three possible structures: **star schema**, **snowflake schema**, or **constellation schema**.

**Star Schema** A star schema consists of one fact table and a set of dimension tables, one per dimension in the conceptual model. Each dimension table has a single attribute as primary key, modeling a one-to-many relationship with the foreign key in the fact table. The primary key attribute is usually a sequentially increasing integer called **surrogate key**, going from 1 to the total number of records in the table. Dates are also often assigned an integer surrogate key in the form *YYYYMMDD*, with the granularity of a day, while the remaining attributes simply specify the generic month/year/quarter/week/etc. the date corresponds to. Degenerate dimensions are either stored as attributes of the fact table, or in a **junk dimension** table, which lists all possible combinations of values of the degenerate dimensions. This structure is called “star” because the fact table is at the center and the dimension tables, which radiate out from it, look like the points of a star.

**Snowflake Schema** In a snowflake schema, some dimension tables are normalized, splitting the data into additional tables. Its advantage is the reduction of redundancy in the dimension tables and therefore a reduction in storage space, however it is often a negligible improvement in comparison to the size of the fact table, and the increased complexity of certain queries

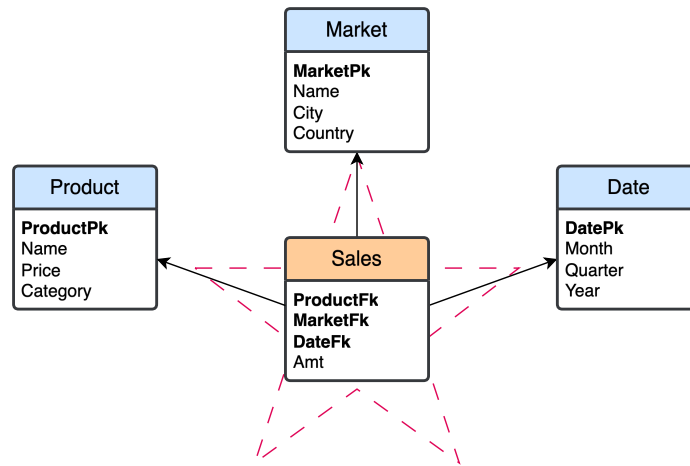


Figure 3.3: Example of a star schema.

that require hierarchies to be traversed. As a result, this schema is not as popular as the star schema.

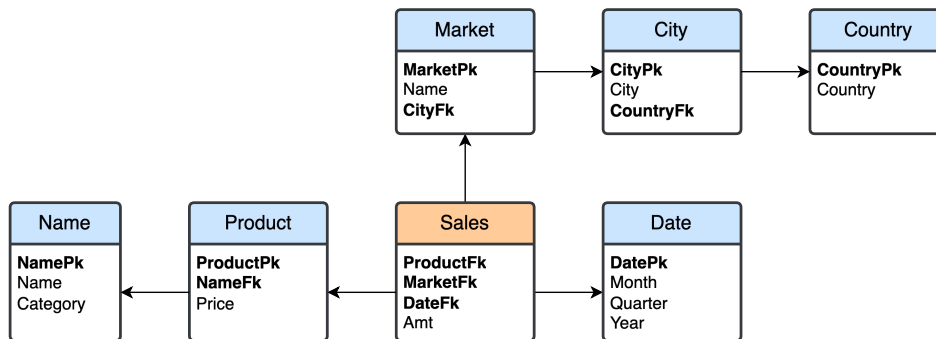


Figure 3.4: Example of a snowflake schema.

**Constellation Schema** A constellation schema is obtained by fusing multiple schemas with one or more dimensions in common, called **conformed dimensions**. The result is a schema with several fact tables that share a number of dimension tables.

## 3.2 Design Approaches

Data warehouse design can be approached from two perspectives: an analysis-driven one, or a data-driven one. In an **analysis-driven** (*bottom-up, metric pull*) approach, data marts are designed based on the data analysis the users want to perform, and then integrated in a single data warehouse. An advantage of this approach is that the focus is to provide what is really needed by the users, rather than what is available, and is usually cheap and fast to implement. A disadvantage is that the data necessary for analysis may not be available, and

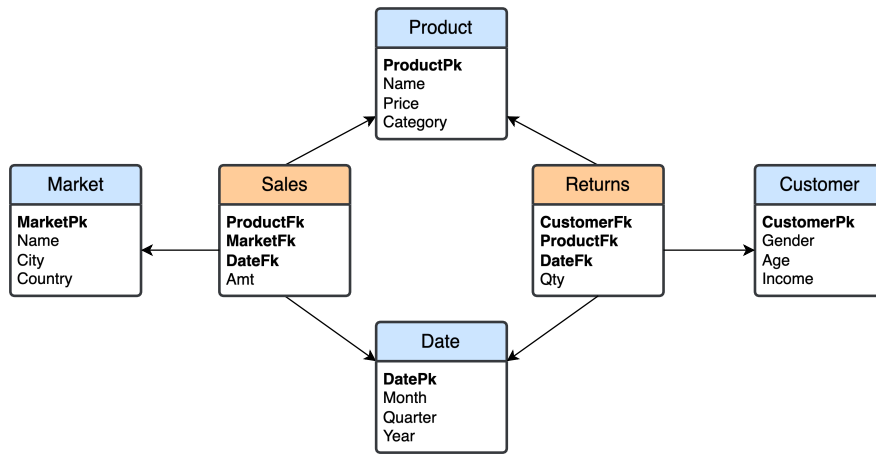


Figure 3.5: Example of a constellation schema.

if a user is too tight on the requirements, the design may miss useful data that is available in the operational system. This approach was first proposed by R. Kimball.

In a **data-driven** (*top-down, data push*) approach, the data warehouse is designed around which data is currently available in the operational system, derivating the data marts afterwards. An initial design can help both users to discover new ways to use the data, and the designers to identify areas on which to focus more. A disadvantage is that it is a generally costly and time-consuming process, and without user involvement, the result may not be useful. This approach was first proposed by W. Inmon.

In practice, these two approaches are used together in different steps. The main phases of data mart design are requirements analysis, initial conceptual design, candidate conceptual design, final conceptual design, and logical design.

**Requirements Analysis** Requirements analysis consists in two sub-phases, requirements gathering and requirements specification. **Requirements gathering** analyzes the nature and purpose of the business, identifying the important business processes and the business questions for which the data warehouse will be used to answer. This is done by directly interviewing the business users and data source system experts. **Requirements specification** uses the business questions found in the previous step to identify the facts, measures, metrics, and dimensions, as well as the granularity of data and dimensional attributes and hierarchies. The specific structure of the worksheet used to organize this information is the following:

- **Business process requirements:** each business question of each process is analyzed individually to find dimensions, measures, and metrics.

| No. | Business questions | Dimensions | Measures | Metrics |
|-----|--------------------|------------|----------|---------|
|-----|--------------------|------------|----------|---------|



- **Fact descriptions:** each identified fact is described in detail, including the preliminary dimensions and measures of the future data mart.

|                          |                        |                      |
|--------------------------|------------------------|----------------------|
| Grain, description, type | Preliminary dimensions | Preliminary measures |
|--------------------------|------------------------|----------------------|

- **Dimensions:** the meaning of each dimension is described, together with its grain.

|      |             |             |
|------|-------------|-------------|
| Name | Description | Granularity |
|------|-------------|-------------|

- **Dimensional attributes:** each non-degenerate dimension is described by a number of attributes. It is good practice to use different names for attributes of different dimensions.

|           |             |
|-----------|-------------|
| Attribute | Description |
|-----------|-------------|

- **Dimensional hierarchies:** any hierarchy between attributes of the same dimension is described, indicating the direction and type (balanced, ragged, or recursive).

|           |                       |                |
|-----------|-----------------------|----------------|
| Dimension | Hierarchy description | Hierarchy type |
|-----------|-----------------------|----------------|

- **Dimensional attribute changes:** if a dimension has changing attributes, these are highlighted separately and annotated with the type (slowly changing of type 1, 2, 3, or fast changing/type 4).

|      |                    |                      |
|------|--------------------|----------------------|
| Name | Changing attribute | Treatment of changes |
|------|--------------------|----------------------|

- **Measures and metrics:** each fact measure has a description, an aggregability type (e.g., additive, semi-additive, etc.) and a flag that indicates if it is calculated from other measures.

|         |             |               |            |
|---------|-------------|---------------|------------|
| Measure | Description | Aggregability | Calculated |
|---------|-------------|---------------|------------|

- **Descriptive attributes of the facts:** contains each descriptive attribute of the identified facts.

|           |             |
|-----------|-------------|
| Attribute | Description |
|-----------|-------------|

- **Summary of dimensions and measures (optional):** if the requirements concern multiple facts, a summary table is used to track which dimensions and measures are associated with which fact. The summary table will have one column per dimension, working as a binary flag to indicate belonging.

|           |        |     |          |
|-----------|--------|-----|----------|
| Dimension | Fact 1 | ... | Fact $n$ |
|-----------|--------|-----|----------|

|         |        |     |          |
|---------|--------|-----|----------|
| Measure | Fact 1 | ... | Fact $n$ |
|---------|--------|-----|----------|

**Conceptual Design** The initial, analysis-driven conceptual design focuses on the user requirements; the candidate, data-driven design is built by considering the data actually available in the operational database. A key sub-phase of this data-driven step is **entity classification**, in which the tables of the operational database are classified into three possible categories:

- **Transaction entities** are tables with records that represent events of interest that occur at a specific point of time and contain measurements or quantities that may be summarized.
- **Component entities** are tables related to a transaction entity via a *one-to-many* relationship. They define the details of each transaction event, so they are useful to answer the 5W-1H questions of a business event.
- **Classification entities** are tables related to a component entity via a (chain of) *one-to-many* relationship(s). They usually represent some hierarchy embedded in the data schema. Their more interesting attributes are collapsed into component entities to then define the dimensional attributes and hierarchies of the data mart.

The final design is obtained by comparing the previous two and finding a compromise between what is *useful* and what can be *delivered*.

**Logical Design** First, each final conceptual data mart design is translated into a relational schema, choosing the most appropriate structure (star or snowflake); then, if multiple data marts are designed, they are integrated in a single constellation schema, evaluating which facts and dimensions will be standardized and/or shared. Ideally, the translation must optimize query performance and maintain an intuitive user interface. Normalizing dimension tables, as mentioned in the previous sections, would typically interfere with both objectives. This phase also explicitly deals with changing dimensions. Type 1 changes simply cause an overwrite of the old value with the new value.

Type 2 causes a new record to be inserted in the dimension table, with its unique surrogate key. This solution, however, creates a problem for analysis that requires counting the number of distinct entities: e.g., if a customer changes address, and we want to count the number of unique customers for some condition with a *COUNT (DISTINCT CustomerFk)*, that

customer will be counted twice. The solution consists in adding an attribute to the dimension table with a value that tracks which records with different surrogate keys actually refer to the same entity. This can be done in three ways:

- By using some “natural” key of the entity, e.g., SSN, student ID, etc;
- By setting the value of the additional attribute to the surrogate key of the first record assigned to the entity;
- By storing the initial value of the surrogate key directly in the fact table, as a degenerate dimension, in order to reduce the need for joins with the dimension table; in turn, this increases the size of the fact table, and may require the usage of a separate mapping table in the staging area to populate the fact table.

Type 3 requires adding two new attributes in the dimension table, one for the new value and one for the modification date; if multiple modifications are done, new records are added to the table, like in Type 2. By comparing the values of current and old attribute values, it is possible to reconstruct the chain of changes applied to that attribute.

Type 4 (fast changing) can be handled in two ways depending on the size of the dimension. If it is small, the technique of Type 2 can be applied, adding a new record with the new value and keeping track of the original primary key. A more appropriate approach, used if the dimension is large, is to break off the fast changing attributes into one or more **mini-dimensions**. The dimension will then be stored in multiple separate tables, one which will contain the non-changing or slowly changing attributes, and one per attribute that changes frequently. If the fast changing attributes are of numerical nature, they are often accompanied by additional attributes that define ranges or hierarchies (e.g., years of service in a company can be grouped in ranges 0-5, 6-10, etc., or as categories like junior, senior, etc.).

Another aspect to consider is how to translate hierarchies. Simple hierarchies that involve only one dimension are directly transformed in dimension tables, where the attributes that form the hierarchy will respect a functional dependency. **Shared hierarchies** (i.e., shared among multiple dimensions) are translated as a single table that is connected multiple times (one per dimension) to the fact table. The fact table will have one foreign key per dimension, to differentiate the relationships. **Similar hierarchies** between dimensions that are very similar but not identical are handled with views of one table that is then shared with the other(s).

**Recursive hierarchies** require special care. The most intuitive solution is to have one table for the dimension that includes a foreign key to itself. Value *NULL* is used for the elements at the top of the hierarchy. This solution is not very efficient, since in order to reconstruct the hierarchical relationships, several joins are needed; additionally, unless we know *a priori* how many elements belong to the hierarchy, there is no way to code the

queries in standard SQL. Some variants of SQL allow for recursive queries, but they are still very slow to execute.

The alternative is to eliminate the foreign key in the dimension table and build an auxiliary table that lists all the hierarchy links between entities, even indirect ones. This table will then contain one row for each link, with three columns: the first two list the primary keys of the linked entities, while the third is an integer that tracks the number of arcs in the path that connects the first entity to the second one. The table can be enriched with additional columns to store whether the link can be further extended to the top or the bottom.

A final aspect to consider is **multivalued dimensions**. There are four possibilities:

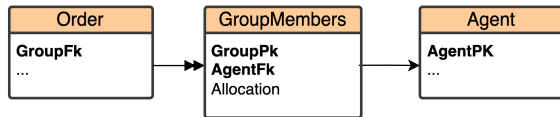
- The fact granularity is changed (Fig 3.6a): instead of using one record for each fact instance, one record is used for each dimension instance. For example, if a product is sold by two agents, the fact table will record one row per agent sale, using a dedicated attribute to store the agent “group” that completed the sale. This increases the memory occupied by the fact table, by a factor equal to the average number of dimension instances “participating” in a single original fact.
- An auxiliary table, called **bridge table**, is added to connect the fact table and the dimension table (Fig. 3.6b). This bridge table will contain one row per fact-dimension pair; in the agent example, if a product is sold by two agents, the bridge table will have two rows, one for each agent, pairing their primary key with the product primary key they jointly sold. This solution violates the properties of a star schema, and may not be accepted by some BI systems.
- An auxiliary table with a *one-to-many* relationship with the fact table is added (Fig. 3.6c). The auxiliary table has one attribute that stores the group, and another that stores a foreign key to the dimension table. This solution also permits additional attributes, such as *allocation*, which indicates the percentage of participation of each dimension instance: e.g., two agents who collaborated may have worked equally and thus have an allocation of 50% each, or one worked more and the workload was split 70%-30%.
- Two auxiliary tables are used, with a middle one acting as a bridge table between the other and the dimension table (Fig. 3.6d). The first table only stores one row per group, with a dedicated primary key, while the second one associates each entity of the dimension with a group. This solution is usually accepted by BI systems.



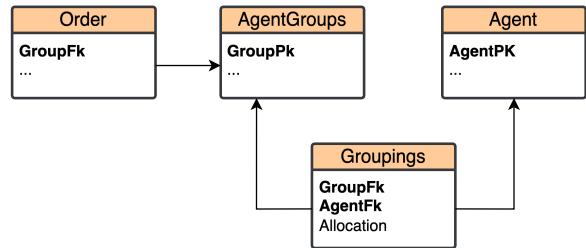
a) New fact granularity.



b) Bridge table.



c) Auxiliary table with a one-to-many relationship.



d) Two auxiliary tables.

Figure 3.6: Possible solutions for multivalued dimensions.

# Chapter 4

## Customer Relationship Management

Customer Relationship Management (CRM) is a set of processes and systems supporting a business strategy to build long-term, profitable relationships with specific customers. The CRM framework can be split into **operational CRM** and **analytical CRM**: the former refers to the automation of customer-facing processes (e.g., Marketing, Sales, Services), while the latter focuses on supporting decision making via the analysis of customer characteristics and behaviours. This chapter will introduce some key ideas behind CRM and show some examples on how to design a data warehouse for CRM applications.

### 4.1 Types of Analyses

The analytical CRM system provides useful information extracted by the analysis of data collected by the operational CRM system, and properly organized in a data warehouse based on the needs of the decision makers. The objective is to find both the best customers for more profitable relationships, and the customers with a high defection risk, i.e., who are more likely to interrupt their relationship with the company.

A **CRM taxonomy** consists in a set of CRM analysis categories, their business questions, and KPIs used to measure the organization's performance. A typical CRM taxonomy includes the following categories:

- **Sales and Marketing Analysis:** the analysis of product sales (by product, sales channel, geographic area, customer, etc.) to identify trends and patterns in order to plan and evaluate the effects of promotional campaigns.
- **Profitability Analysis:** revenues from sales must be analyzed by taking into account the costs of production, marketing, and nonconformance. Profitability of sales is an indicator of the company's ability to maintain an advantage over competitors.
- **Service Quality Analysis:** the success of a company also depends on how well customer demands and expectations are met. Analysis in this category is focused on finding

the most common reasons for customer complaints or returns, as well as determining the ability to deliver orders fully and on time.

- **Customer Analysis:** analysis of data on customers is used to group them based on their attributes, find out their spending habits and if/when their behaviour changes. Specific analyses include customer segmentation analysis, retention analysis, attrition (churn) analysis, and satisfaction analysis.

Below are some examples of data warehouse for each category.

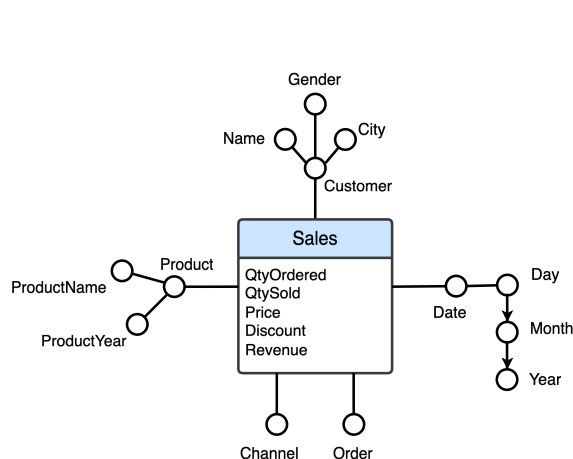


Figure 4.1: Data warehouse for Sales and Marketing Analysis.

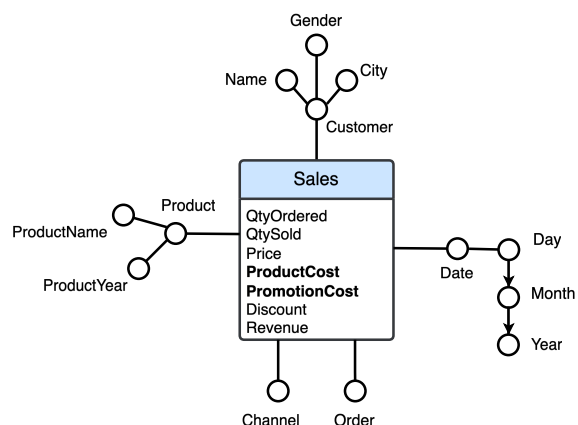


Figure 4.2: Data warehouse for Profitability Analysis. This one is obtained by adding the appropriate cost measures to the fact table of the previous example.

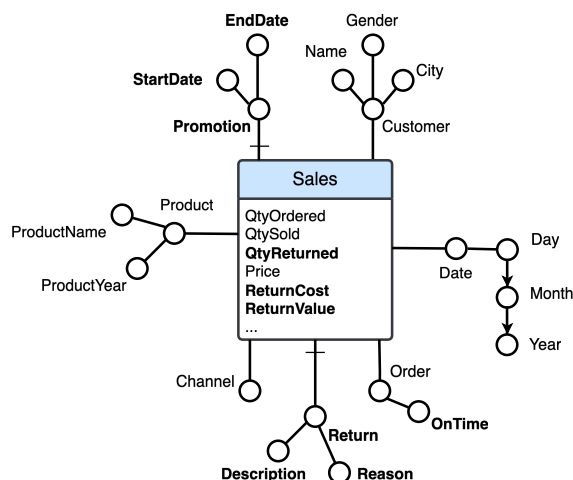


Figure 4.3: Data warehouse for Service Quality Analysis. Here, both new measures and new dimensions are introduced to the schema.

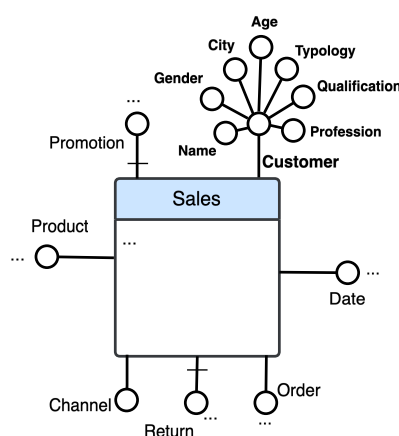


Figure 4.4: Data warehouse for Customer Analysis. This schema is obtained by adding customer-related dimensional attributes to *customer*.

Other than the *sales* fact table, each data warehouse will likely contain other fact tables. For example, service quality analysis may require a *returns* fact table to track product returns and analyze them by date, product, order, reason, etc. Similarly, customer analysis could include a *customerTypology* fact table that not only tracks the category the customer belongs to (e.g., new, constant, occasional, inactive), but also any changes over time.



# Chapter 5

## Multidimensional Data Cube Model

This chapter introduces the multidimensional (data) cube model, another logical model used in data warehousing to analyze data in aggregated form in an intuitive way. A multidimensional cube model represents facts with  $n$  dimensions as points in an  $n$ -dimensional space; a point is identified by the values of dimensions, and has an associated set of measures. An example of a 3-dimensional cube is shown in Fig. 5.1. The following sections will describe the main operations that can be performed on cubes.

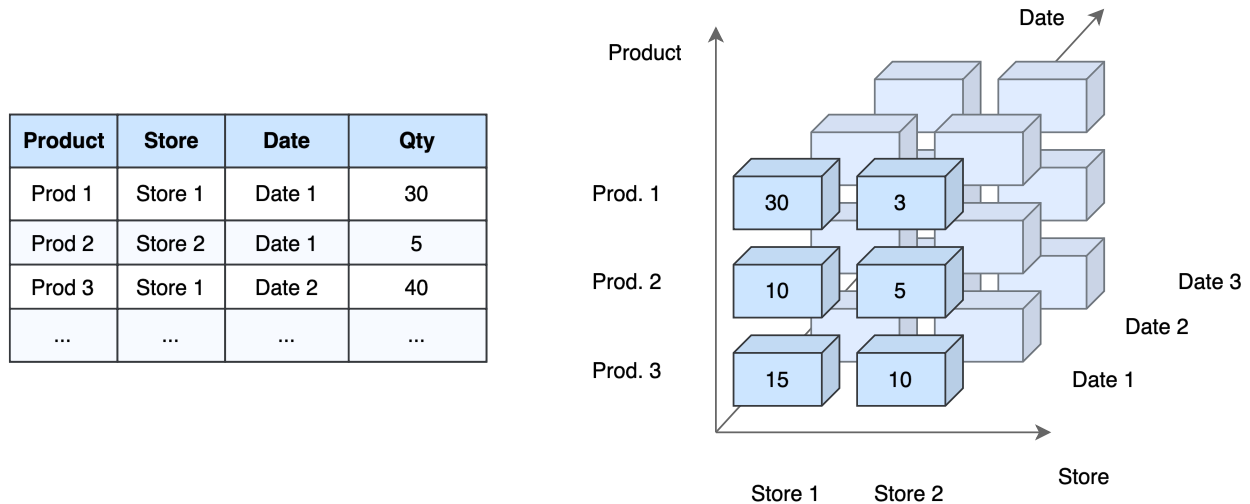


Figure 5.1: Example of a 3-dimensional cube. *Product*, *Date*, and *Store* are dimensions, while *Qty* is a measure of a fact.

### 5.1 OLAP Operations in the Data Cube

There are a few main operators that can be used to manipulate the data cube:

- **Slice** and **dice**: both operators generate sub-cubes without summarizing any information. The *slice* operator selects a cross-section of the cube by “cutting” it across a

dimension at a given value. For example, slicing the cube in Fig. 5.1 along *Date* at value “Date 1” produces the sub-cube that contains all the points in the front face. The *dice* operator is similar to slicing, but it selects a sub-cube along two or more dimensions.

- **Roll-up** and **drill-down**: *roll-up* (also called *drill-up*) summarizes data at different levels of detail, either by reducing dimensions or by climbing dimension hierarchies. If *roll-up* is used on the cube in figure 5.1 on the *Date* dimension, the result is a cube with only two dimensions *Product* and *Store*, where each point contains the *Qty* value aggregated over all date values. The equivalent in SQL is a *group by* done on all other dimensions not involved in the operator. *Drill-down* does the reverse operation, producing more detailed data by adding dimensions or stepping down a dimensional attribute hierarchy. A cube that is rolled-up along all dimensions is called **apex cube** (or 0-dimensional cube).
- **Drill-through**: this operator produces the facts that satisfy a given condition, reported in the form of a relational table.
- **Pivot**: the *pivot* (or *rotate*) operator changes the presentation of the data cube by rotating the axes. It does not affect the actual data stored in the points.

The textual notation used in the data cube model is the following:

- `Sales(Store, Product, Date)` is the cube with dimensions *Store*, *Product*, and *Date*, and some measure(s).
- `Sales(Store, Product, 'D1')` is a slice of the cube at *Date* = 'D1'.
- `Sales(Store, 'P1', 'D1')` is a dice of the cube at *Product* = 'P1' and *Date* = 'D1'.
- `Sales(Store, Product, *)` is a roll-up on the *Date* dimension, applying the sum aggregation function on the measure(s). The asterisk (\*) indicates that the dimension is removed. `Sales(*,*,*)` produces the apex cube.
- `Sales(Store, 'P1', *)` is a combination of roll-up and slice: it slices the cube at *Product* = 'P1' and rolls-up on the *Date* dimension.

## 5.2 The Extended Cube

A cube can be extended with borders made of cells containing the value of aggregate functions applied to the fact measures across each dimension, and combination of dimensions. In the previous example cube, it can be extended like in Fig. 5.2. An extended cube can be seen as a generalization of extended cross-tabulations, where the additional cells contain the results

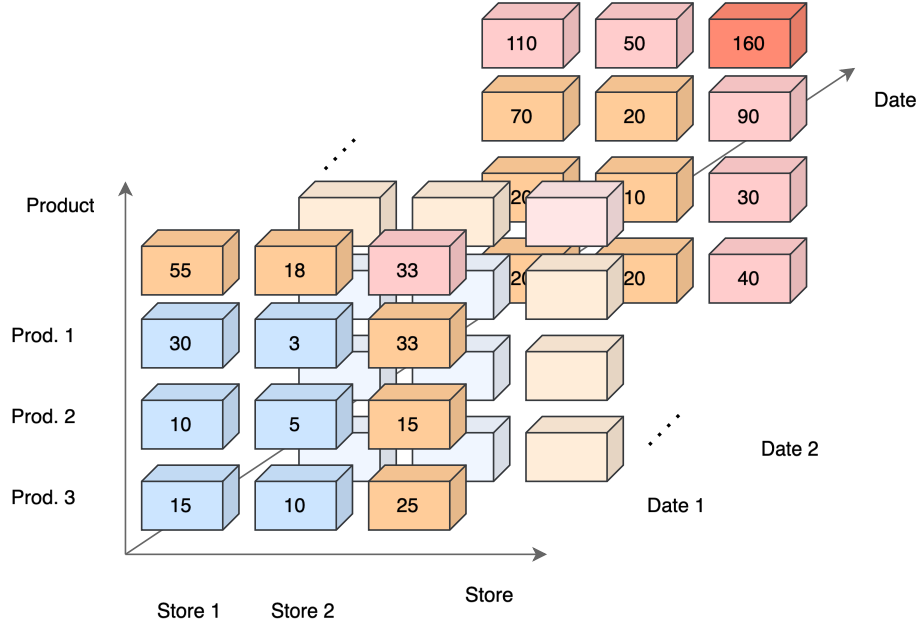


Figure 5.2: Example of an extended cube. The warm colored cells contain the result of aggregation along dimensions. The face in the back contains the apex cube in the top right corner.

of aggregation functions. Whenever a dimension is used as coordinate instead of one of its values, the result is a **cuboid**. For example,  $(\text{Store}, \text{Product}, *)$  is the cuboid obtained by rolling-up by *Date*. To speed up data analysis, data cube systems can pre-compute all or some cuboids and store them as **materialized views**. The total number of possible cuboids for a cube with  $n$  dimensions is  $2^n$ ; the total number of cells is

$$\sum_{i=1}^n (d_i + 1)$$

where  $d_i$  is the number of values for the  $i$ -th dimension. An alternative way to indicate a cuboid is to omit the asterisk altogether:  $(\text{Sales}, \text{Product}, *)$  is equivalent to  $(\text{Sales}, \text{Product})$ .  $(\text{Store}, \text{Product}, \text{Date})$  denotes the whole data cube, while  $()$  denotes the (dimensionless) total sum of all sales.

Cuboids can be represented as a **lattice**, which summarizes the partial order relations between them. We say that cuboid  $C_1$  is below cuboid  $C_2$  ( $C_1 \preceq C_2$ ) if and only if  $C_1$  can be computed from  $C_2$ . In general, the computation of  $C_1$  from  $C_2$  depends on the aggregate function used, which may be distributive, algebraic, or holistic.

# Chapter 6

## On-line Analytical Processing Analysis

The term OLAP refers to multidimensional analysis of large amounts of data. This is usually done with interactive tools with graphical interfaces that allow the user to make requests, which are automatically translated into SQL: for this purpose, SQL has been extended with some operators to support grouping and aggregation using analytic functions. This chapter will give an overview of OLAP systems, SQL data analysis, and introduce reports.

### 6.1 OLAP Systems Solutions

An OLAP client provides graphical environments where the user can click or use drag-and-drop operations to provide input to the system. Experienced users can also write explicit SQL queries. The client can interact with the data manager using one of three possible solutions:

- **DOLAP**: the client interacts with a local DOLAP (Desktop OLAP) system, which manages small amounts of data extracted from the main OLAP server, Data server, or operational DBMS. This is a good choice for users who need to work offline or move frequently (e.g., who travel a lot), or who do not need to perform complex queries. An example of DOLAP system are Excel Pivot Tables and Microsoft Power Pivot.
- **OLAP server**: the client interacts with an OLAP server, communicating via a dedicated API. The server hosts the multidimensional cube vision of a data warehouse; it can be of three types:
  - **MOLAP** (Multidimensional OLAP), which stores both the data cube (taken from a relational Data server) and the aggregates in the extended cube in local memory, using a specialized multidimensional array-based data structure. MOLAP servers do not support SQL; instead, they support proprietary data cube query languages

such as Microsoft's MDX. This solution provides the best performances, but it is not suitable for large amounts of data.

- **ROLAP** (Relational OLAP), which stores both data and aggregates in the Data server. ROLAP servers may implement functionalities not supported by SQL.
- **HOLAP** (Hybrid OLAP), which combines MOLAP and ROLAP approaches: it splits data storage in a MOLAP server and a relational Data server. Splitting can be done in different ways. One method is to store the detailed data in the Data server, and keeping only precomputed, aggregate data in the MOLAP server; another method is to store the most recent aggregate data in the MOLAP server, to provide faster access, and older aggregate data in the Data server.

Data updates are usually done periodically in batches. Requests of the OLAP client to the server are formulated in SQL or other proprietary languages, and the result is typically returned in proprietary formats or in XML.

- **RDBMS** (Relational DBMS): the data warehouse is entirely stored in a relational Data server (i.e., a relational DBMS). All interactions with the OLAP client are directly expressed in SQL. An advantage of this solution is that it uses standard technology already available. In the past, it was considered to be too inadequate, both for performance limitations of RDBMSs, and lack of OLAP support in SQL. As modern RDBMS systems producers have added more and more specialized functionalities dedicated to OLAP applications (**OLAP-aware RDBMS**), this solution has become more and more popular.

All further discussions will assume the RDBMS solution.

## 6.2 Reports

This section is dedicated to reports, showing examples of how SQL can be used to generate the appropriate output for commonly asked business questions. All examples are based on the following table:

*Sales(Customer, Product, Brand, Date, City, Region, Area, Quantity, Revenue, Margin)*

Attributes don't have any null value.

**Analytic SQL** The SQL has been extended to allow the use of several analytic functions, also known as Windows Functions, to facilitate the development of complex data analysis. A query has the following structure:

```

SELECT Select_attr ( $S_A$ ), Select_agg_fun ( $S_{AF}$ ), Analytic_fun ( $A_F$ ) OVER (
    [PARTITION BY <attr_list>]
    [ORDER BY <sort_attr_list>] [window_clause])
FROM Fact_table ( $F$ ) and Dimension_table ( $D$ )
WHERE Where_cond ( $W_C$ )
GROUP BY Grouping_attr ( $G_A$ )
HAVING Having_cond ( $H_C$ ) with Agg_fun ( $H_{AF}$ )
ORDER BY Sort_attr ( $O_A$ );

```

A query written in analytic SQL is processed in this order: first, the operations in FROM, WHERE, GROUP BY, and HAVING clauses are performed; then, the analytic functions in the select are applied to the result of the previous step, producing a new result that adds new columns to the previous; finally, projection and sorting are applied to compute the final result.

Analytic functions are used in the SELECT, with the **OVER** clause; they can either be applied to all the records, or separately to disjoint subsets obtained by partitioning the records by the value of an expression defined on the attributes of the record, using **PARTITION BY**. The partitioning operation is similar to that of a GROUP BY, but it does not output a single record for each group: instead, it produces as many records as it receives in input, extended with the new calculated attributes. If PARTITION BY is omitted, the set of records acts as a single group.

### 6.2.1 Simple Reports

Simple reports are obtained from simple SQL queries. For example, take the following business question: "What was the total revenue of sales for the month of January 2025, by brand and by product?". The SQL query to answer this question and produce the correct report is:

```

SELECT Brand, Product, SUM(Revenue) AS Total_Revenue
FROM Sales
WHERE YEAR(Date) = 2025 AND MONTH(Date) = 1
GROUP BY Brand, Product
ORDER BY Brand, Product;

```

Selection and projection are used to express slices and dices. Roll-ups and drill-downs are expressed using grouping: the former are obtained by removing attributes from the grouping clause, while the latter are obtained by adding attributes.

**ROLLUP** Suppose now we want to get a report with subtotals showing the total revenue from sales in the year 2025, by brand and by product, including the total revenue for each brand, and the overall total revenue. This can be done with standard SQL, although it requires three separate queries combined with *UNION ALL*:

```

SELECT Brand, Product, SUM(Revenue) AS Revenue
FROM Sales
WHERE YEAR(Date) = 2025
GROUP BY Brand, Product

```

**UNION ALL**

```

SELECT Brand, NULL AS Product, SUM(Revenue) AS Revenue
FROM Sales
WHERE YEAR(Date) = 2025
GROUP BY Brand

```

**UNION ALL**

```

SELECT NULL AS Brand, NULL AS Product, SUM(Revenue) AS Revenue
FROM Sales
WHERE YEAR(Date) = 2025
ORDER BY Brand, Product;

```

The first query computes the revenue by brand and product, the second computes the subtotals for each brand, and the third computes the overall total. Fig. 6.1 shows an example of what the report would graphically look like.

| Revenue by Brand and Product |              |             |
|------------------------------|--------------|-------------|
| Brand                        | Product      | Revenue     |
| B1                           | P1           | 1000        |
|                              | P2           | 1500        |
|                              | P3           | 2000        |
| <b>B1</b>                    | <b>Total</b> | <b>4500</b> |
| B2                           | P1           | 1200        |
|                              | P3           | 1000        |
|                              | P4           | 800         |
| <b>B2</b>                    | <b>Total</b> | <b>3000</b> |
| <b>Total</b>                 |              | <b>7500</b> |

Figure 6.1: Report with subtotals for each brand.

Computing all these queries independently is inefficient: for this reason, many DBMSs support the **ROLLUP** clause, which automatically computes the totals and subtotals across the specified grouping attributes. Using **ROLLUP**, the previous query can be simplified as:

```

SELECT Brand, Product, SUM(Revenue) AS Revenue
FROM Sales
WHERE YEAR(Date) = 2025
GROUP BY ROLLUP(Brand, Product)

```

**ORDER BY** Brand, Product;

The order in which attributes are specified in the ROLLUP clause is important, as it determines the order in which aggregates are computed. One way to interpret ROLLUP is as an operation that computes a path through the lattice of cuboids, starting from the most detailed one (i.e., the one with all attributes on the clause), and then removing one attribute at a time, in the reverse order in which they are specified in the ROLLUP clause, until no attribute is left. In this example, the order is:  $(Brand, Product)$ ,  $(Brand)$ ,  $()$ .

**CUBE** Suppose we want to get a report similar to the one in Fig. 6.1, but in addition, we want to add totals for each row and each column, meaning that we need to find the total revenue also for each product. Also in this case, a solution with standard SQL would require multiple queries combined, so it is better to use the **CUBE** clause included in many DBMSs. The query is in Fig. 6.2.

```
SELECT Brand, Product, SUM(Revenue) AS Revenue
FROM Sales
WHERE YEAR(Date) = 2025
GROUP BY CUBE(Brand, Product)
ORDER BY Brand, Product;
```

Unlike roll-up, the order of the attributes does not matter. The report generated by this query is shown in Fig. 6.2.

| Revenue by Brand and Product |                 |             |
|------------------------------|-----------------|-------------|
| Brand                        | Product         | Revenue     |
| B1                           | P1              | 1000        |
|                              | P2              | 1500        |
|                              | P3              | 2000        |
| <b>B1</b>                    | <b>Total</b>    | <b>4500</b> |
| B2                           | P1              | 1200        |
|                              | P3              | 1000        |
|                              | P4              | 800         |
| <b>B2</b>                    | <b>Total</b>    | <b>3000</b> |
|                              | <b>Total P1</b> | 2200        |
|                              | <b>Total P2</b> | 1500        |
|                              | <b>Total P3</b> | 3000        |
|                              | <b>Total P4</b> | 800         |
| <b>Total</b>                 |                 | <b>7500</b> |

Figure 6.2: Report with subtotals for each brand and each product.

More than one ROLLUP or CUBE clause can be used in the same GROUP BY statement. For example, if we write:



**GROUP BY ROLLUP(A), ROLLUP(B, C)**

the query will compute the cartesian product of both ROLLUPs; the first is  $\{(A), ()\}$ , and the second is  $\{(B,C), (B), ()\}$ , so their combination yields the following grouping sets:

$\{(A, B, C), (A, B), (A), (B,C), (B), ()\}$

This is also allowed for combinations of simple groupings with ROLLUPs and CUBEs, although in this case the result is constrained by the grouping attributes. For example, if we write:

**GROUP BY A, ROLLUP(B, C)**

the result will be  $\{(A, B, C), (A, B), (A)\}$ , because A must appear in all groupings. If we write:

**GROUP BY A, CUBE(B, C)**

the result will be  $\{(A, B, C), (A, B), (A, C), (A)\}$ .

## 6.2.2 Moderately Difficult Reports

Some commonly requested reports cannot be expressed easily, as they require comparisons. An example is: “What are the total revenues in 2025 by brand and by product, with the percentage change from the previous year?”. One way to write the corresponding SQL query is to write two separate queries, one for the year 2025 and one for 2024, which individually find the total revenues grouping by brand and product. Then, if the same products are sold in both years, the final result can be found performing a natural join of the two results; otherwise, a full join is required. To write the full query, there are two options: either the subqueries are written in the FROM clause, or they are specified before the main one using the WITH clause. Using WITH, the query is:

```
WITH Revenue25 AS(
    SELECT Brand, Product, SUM(Revenue) AS TotRevenue25
    FROM Sales
    WHERE YEAR(Date) = 2025
    GROUP BY Brand, Product
),
Revenue24 AS(
    SELECT Brand, Product, SUM(Revenue) AS TotRevenue25
    FROM Sales
    WHERE YEAR(Date) = 2024
    GROUP BY Brand, Product
)

SELECT Revenue25.Brand AS Brand, Revenue25.Product AS Product,
TotRevenue25, TotRevenue24,
```

```
ROUND(100*(TotRevenue25 - TotRevenue24)/TotRevenue25) AS Delta
FROM Revenue24 FULL JOIN Revenue25 USING(Brand, Product)
ORDER BY Brand, Product;
```

For simplicity, we omitted the handling of cases where the total revenue for a given product is NULL is a year it was not sold in; it would need an additional CASE statement.

Other business questions may require the use of OVER and PARTITION BY.

### 6.2.3 Very Difficult Reports

An example of a business question that needs a very difficult report to be answered is: “What are the top 5 best products sold in a given region?”. This example motivates the introduction of an additional operator in analytic SQL, called RANK.

**RANK and DENSE\_RANK** The RANK function is used to sort the records in a set based on the value of an attribute or an expression (i.e., an aggregate function), assigning a position in a ranking to each record. The rank of a value is defined as 1 plus the number of other values that strictly precede it. If  $k > 1$  values are equal, they are assigned the same rank, and the rank assigned to the next value is also increased by that  $k$ . For example, if the values are (1, 2, 2, 3), their ranks are (1, 2, 2, 4). The standard order is ascending, meaning that rank 1 is associated with the record with the lowest value, but the ordering can be reversed. This function is specified in the SELECT clause as follows:

```
<RankFun>()
OVER ( [PARTITION BY <attr_list>]
ORDER BY <sort_attr_list> ) [AS Ide]
```

Partitioning is optional. If it is omitted, all records are assumed to be part of the same partition set, so that each record is assigned a unique rank. If partitioning is used, ranking is done by grouping by the specified records and only within each group. The ORDER BY is mandatory, since ranking also requires the data to be sorted beforehand. DENSE\_RANK is a variant of RANK where the rank of a value is defined as 1 plus the number of distinct values that strictly precede it. Using the previous example, the values (1, 2, 2, 3) would have ranks (1, 2, 2, 3). If the collection of values contains no duplicates, RANK and DENSE\_RANK produce the same result. Other related functions include:

- **PERCENT\_RANK**: defined as  $\text{RANK} - 1$  divided by  $\text{number of rows} - 1$ . It's a normalized version of RANK, since it always produces values in the range  $[0, 1]$ .
- **ROW\_NUMBER**: simply returns a sequential number for each record, starting from 1.
- **CUME\_DIST**: it returns a value between 0 and 1 to each record according to the number of records lower than or equal to it. In other words, it calculates the empirical

cumulative distribution of the values in the collection. Equal values are assigned to the same number.

- **NTILE(*n*)**: returns the *n*-tile of the collection of records, i.e., a partition of *n* groups each with the same number of records as the others (eventually plus or minus one if a perfect split is not possible), assigning to each record the number of the corresponding group.

**LAG and LEAD** The LAG and LEAD functions are used to access data from rows that come before or after the current row in the result set, respectively. The syntax is:

```
LAG(<attr>, <offset>, <default>)  
LEAD(<attr>, <offset>, <default>)
```

The first argument is the attribute from which the value is retrieved from. The second argument is the number of rows to skip above or below. The last argument is a default value to be used if the stored value in the specified row is empty (e.g., if the offset is larger than the number of rows available). Only the first argument is mandatory; the offset is otherwise set to 1, and the default is set to NULL.

**Window Functions** Windows are used to make calculations that take into account a set of rows related to the current one. They can be used to compute cumulative, moving, and centered aggregates. The window size can be determined in a physical way with the **ROWS** option, or in a logical way using the **RANGE** option. Commonly, windows are defined over the date attribute. The window can include all records in the set on which it is defined, or only the current record. For example, to calculate a cumulative sum, the first record is fixed and the end of the window moves from first to last record. A moving average requires both window extremes to move. The syntax used to define a window is:

```
<agg_fun>(<expr>)  
OVER (  
    [PARTITION BY <attr_list>]  
    [ORDER BY <sort_attr_list> [<ROWS or RANGE> <window_clause>]]  
) [AS Ide]
```

Examples of window specifications are:

- **ROWS UNBOUNDED PRECEDING**. The window begins with the first record of the partition and ends with the current record.
- **ROWS BETWEEN 5 PRECEDING AND 5 FOLLOWING**. The window includes all records that are within 5 rows before and after the current record.
- **RANGE BETWEEN INTERVAL 5 DAYS PRECEDING AND CURRENT ROW** (the date attribute is used). The window includes all records whose data falls within 5 days before the date of the current one.

# Index

Data Model, 10

Data Warehouse, 3

Data, Information, Knowledge, 2

Entity, Property, Relationship, 9

Functional Dependency, 6

Information System, 3

Key, 14

Relation, 14

Relational Database, 13

Symbolic Model, 8

Tuple, 13